

Hive Assignment Report

- Assignment: Hive Internal & External Tables + SCD Type 2
- Technologies Used:
 - Apache Hive
 - Hadoop HDFS
 - Docker
 - PostgreSQL Metastore
 - OpenCSVSerde

1. Assignment Objective

The objective of this assignment was to practice working with Apache Hive in a realistic data engineering environment. The assignment focused on:

1. Creating internal and external Hive tables.
2. Loading CSV data into Hive.
3. Solving delimiter issues inside CSV values.
4. Understanding the difference between managed and external tables.
5. Implementing Slowly Changing Dimension Type 2 (SCD Type 2).
6. Building an SCD Type 2 solution without using transactional tables, UPDATE, or DELETE operations.

The assignment was implemented using Docker on Windows with the `itversity/itvdelab` environment.

2. Project Structure

The project was organized in a clean GitHub-friendly structure to separate source data, SQL scripts, and outputs.

```
hive-assignment/
├── data/
│   ├── customer_scd2_mixed.csv
│   └── customer_updated.csv
├── output/
│   └── customer_scd2_mixed.csv
├── sql/
│   ├── 01_create_customer_tables.sql
│   ├── 02_drop_table_test.sql
│   └── 03_scd2_without_update_delete.sql
└── docker-compose.yml
```

└─ README.md

This structure helped separate:

- Source files
- Hive SQL scripts
- Final generated outputs
- Infrastructure configuration

3. Docker Environment Setup

A separate Docker environment was created specifically for Hive to avoid conflicts with another NiFi assignment running on the same machine.

The environment included:

- Hive
- Hadoop HDFS
- YARN
- PostgreSQL Hive Metastore

Docker Compose Design

A dedicated `docker-compose.yml` file was used for the Hive environment.

Why a Separate Environment?

I intentionally separated the Hive assignment from the NiFi assignment for the following reasons:

- Avoid port conflicts.
- Avoid service dependency issues.
- Improve project organization.
- Simulate real enterprise environment isolation.
- Make GitHub repository cleaner and easier to understand.

4. HDFS Data Preparation

The provided CSV files were uploaded into HDFS using the following directories:

```
/user/hive/customer/scd2_source  
/user/hive/customer/updated
```

/user/hive/output

The source files used were:

- customer_scd2_mixed.csv
- customer_updated.csv

```
TERMINAL 27 OUTPUT DEBUG CONSOLE PORTS
PS C:\Users\moham\Desktop\week 8> docker exec -it hive_assignment bash
itversity@hive_assignment:~$ hdfs dfs -rm -r /user/hive/customer
Deleted /user/hive/customer
itversity@hive_assignment:~$ hdfs dfs -rm -r /user/hive/output
Deleted /user/hive/output
itversity@hive_assignment:~$ hdfs dfs -mkdir -p /user/hive/customer/scd2_source
itversity@hive_assignment:~$ hdfs dfs -mkdir -p /user/hive/customer/updated
itversity@hive_assignment:~$ hdfs dfs -mkdir -p /user/hive/output
itversity@hive_assignment:~$ hdfs dfs -ls /user/hive/customer/scd2_source
itversity@hive_assignment:~$ hdfs dfs -put -f /opt/hive_data/customer_scd2_mixed.csv /user/hive/customer/scd2_source/
itversity@hive_assignment:~$ hdfs dfs -ls /user/hive/customer/scd2_source
Found 1 items
-rw-r--r--  1 itversity supergroup    28979 2026-05-09 23:07 /user/hive/customer/scd2_source/customer_scd2_mixed.csv
itversity@hive_assignment:~$ hdfs dfs -ls /user/hive/customer/updated
itversity@hive_assignment:~$ hdfs dfs -put -f /opt/hive_data/customer_updated.csv /user/hive/customer/updated/
itversity@hive_assignment:~$ hdfs dfs -ls /user/hive/customer/updated
Found 1 items
-rw-r--r--  1 itversity supergroup    513165 2026-05-09 23:08 /user/hive/customer/updated/customer_updated.csv
itversity@hive_assignment:~$
```

5. Creating Internal and External Tables

Two types of Hive tables were created:

5.1 External Table

The external table was created using a custom HDFS location.

Purpose

The external table separates:

- Table metadata
- Physical raw files in HDFS

This means deleting the table does not delete the original data files.

5.2 Internal Table

The internal table was created as a managed Hive table.

Purpose

Hive fully manages:

- Metadata
- Physical data files

Deleting the internal table removes both metadata and data.

```
itversity@hive_assignment:~$ hive -f /opt/hive_sql/01_create_customer_tables.sql
```

```
hive> SHOW CREATE TABLE ext_customer;
OK
CREATE EXTERNAL TABLE `ext_customer`(
  `customer_id` string COMMENT 'from deserializer',
  `name` string COMMENT 'from deserializer',
  `email` string COMMENT 'from deserializer',
  `phone_number` string COMMENT 'from deserializer',
  `address` string COMMENT 'from deserializer',
  `join_date` string COMMENT 'from deserializer',
  `start_date` string COMMENT 'from deserializer',
  `end_date` string COMMENT 'from deserializer',
  `is_current` string COMMENT 'from deserializer')
ROW FORMAT SERDE
  'org.apache.hadoop.hive.serde2.OpenCSVSerde'
WITH SERDEPROPERTIES (
  'escapeChar'='\\',
  'quoteChar'='"',
  'separatorChar'=',')
STORED AS INPUTFORMAT
  'org.apache.hadoop.mapred.TextInputFormat'
OUTPUTFORMAT
  'org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat'
LOCATION
  'hdfs://localhost:9000/user/hive/customer/scd2_source'
TBLPROPERTIES (
  'bucketing_version'='2',
  'skip.header.line.count'='1',
  'transient_lastDdlTime'='1778368426')
Time taken: 1.312 seconds, Fetched: 26 row(s)
```

```
hive> SHOW CREATE TABLE int_customer;
OK
CREATE TABLE `int_customer` (
  `customer_id` string COMMENT 'from deserializer',
  `name` string COMMENT 'from deserializer',
  `email` string COMMENT 'from deserializer',
  `phone_number` string COMMENT 'from deserializer',
  `address` string COMMENT 'from deserializer',
  `join_date` string COMMENT 'from deserializer',
  `start_date` string COMMENT 'from deserializer',
  `end_date` string COMMENT 'from deserializer',
  `is_current` string COMMENT 'from deserializer')
ROW FORMAT SERDE
  'org.apache.hadoop.hive.serde2.OpenCSVSerde'
WITH SERDEPROPERTIES (
  'escapeChar'='\\',
  'quoteChar'='"',
  'separatorChar'=',')
STORED AS INPUTFORMAT
  'org.apache.hadoop.mapred.TextInputFormat'
OUTPUTFORMAT
  'org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat'
LOCATION
  'hdfs://localhost:9000/user/hive/warehouse/int_customer'
TBLPROPERTIES (
  'bucketing_version'='2',
  'transient_lastDdlTime'='1778368433')
Time taken: 0.062 seconds, Fetched: 25 row(s)
```

```
hive> SELECT customer_id, name, address, join_date, start_date, end_date, is_current FROM ext_customer LIMIT 5;
OK
16324 Jennifer Harris 323 Yoder Place East Katherinemouth, NM 43638 2/3/2025 4/1/2025 1
13094 Lisa Morris 85587 Cook Flats Jamesfurt, AS 96155 3/6/2025 4/1/2025 1
13483 Jesus Cochran 57000 Nancy Crescent Apt. 310 Allenborough, CT 33116 1/18/2025 4/1/2025 1
13819 Jacqueline Russo 827 Janet Port Apt. 218 Espinozafort, PW 44894 1/9/2025 4/1/2025 1
16871 Curtis Smith 4373 Gordon Tunnel Carmenshire, NE 95804 1/10/2025 4/1/2025 1
Time taken: 1.032 seconds, Fetched: 5 row(s)
hive> SELECT customer_id, address, length(address) AS address_length, is_current FROM ext_customer LIMIT 5;
OK
16324 323 Yoder Place East Katherinemouth, NM 43638 45 1
13094 85587 Cook Flats Jamesfurt, AS 96155 36 1
13483 57000 Nancy Crescent Apt. 310 Allenborough, CT 33116 52 1
13819 827 Janet Port Apt. 218 Espinozafort, PW 44894 46 1
16871 4373 Gordon Tunnel Carmenshire, NE 95804 40 1
Time taken: 0.443 seconds, Fetched: 5 row(s)
hive> █
```

6. CSV Delimiter Problem

Problem Description

The address column contained commas inside the actual address values.

Example:

```
"123 Main St, New York"
```

Hive originally interpreted the comma inside the address as a column separator.

This caused:

- Broken rows
- Shifted columns
- Incorrect data loading

Solution

After researching Hive CSV parsing methods, I found that `OpenCSVSerde` is the best solution for handling quoted CSV values.

The following SerDe configuration was used:

```
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES (  
    "separatorChar" = ",",  
    "quoteChar"     = "\"",  
    "escapeChar"    = "\\"  
)
```

Why OpenCSVSerde?

`OpenCSVSerde` allows Hive to:

- Ignore commas inside quoted values.
- Correctly parse CSV files.
- Preserve the original separator without modifying the source file.

This was important because the assignment explicitly required solving the delimiter issue without changing the separator.

```
CREATE EXTERNAL TABLE `ext_customer`(  
  `customer_id` string COMMENT 'from deserializer',  
  `name` string COMMENT 'from deserializer',  
  `email` string COMMENT 'from deserializer',  
  `phone_number` string COMMENT 'from deserializer',  
  `address` string COMMENT 'from deserializer',  
  `join_date` string COMMENT 'from deserializer',  
  `start_date` string COMMENT 'from deserializer',  
  `end_date` string COMMENT 'from deserializer',  
  `is_current` string COMMENT 'from deserializer')  
ROW FORMAT SERDE  
  'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES (  
  'escapeChar'='\\',  
  'quoteChar'='\"',  
  'separatorChar'=',')  
)
```

7. Dropping Internal and External Tables

The assignment required testing the behavior of dropping both table types.

7.1 Dropping the Internal Table

After dropping the internal table:

- Hive metadata was deleted.
- Physical files were also removed from HDFS.

This confirmed that managed tables are fully controlled by Hive.

7.2 Dropping the External Table

After dropping the external table:

- Hive metadata was deleted.
- The original CSV files remained inside HDFS.

This confirmed that external tables protect raw source data.

My Observation

External tables are more suitable for enterprise data lake architectures because raw data remains independent from Hive metadata.

```
PS C:\Users\moham\Desktop\week 8> docker exec -it hive_assignment bash
itversity@hive_assignment:~$ hive -f /opt/hive_sql/02_drop_table_test.sql
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/opt/apache-hive-3.1.2-bin/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/opt/hadoop-3.3.0/share/hadoop/common/lib/slf4j-log4j12-1.7.25.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Hive Session ID = 5f22b2de-ffac-4382-8d18-db7e6eae6fb5

Logging initialized using configuration in jar:file:/opt/apache-hive-3.1.2-bin/lib/hive-common-3.1.2.jar!/hive-log4j2.properties Async: true
Hive Session ID = 312ea86a-4b1b-4375-b5e5-3fc5785ec0a5
OK
Time taken: 1.715 seconds
OK
Time taken: 0.128 seconds
itversity@hive_assignment:~$ hdfs dfs -ls /user/hive/customer/scd2_source
Found 1 items
-rw-r--r--  1 itversity supergroup      28979 2026-05-09 23:07 /user/hive/customer/scd2_source/customer_scd2_mixed.csv
itversity@hive_assignment:~$
```

8. Slowly Changing Dimension Type 2 (SCD Type 2)

Goal

The goal was to maintain historical customer records while also tracking the latest customer information.

The following files were used:

- customer_scd2_mixed.csv → Existing customer dimension
- customer_updated.csv → New incoming updates

Business Logic

For changed customers:

- Old record:
 - is_current = 0
 - end_date = current_date
- New record:
 - is_current = 1
 - end_date = NULL

For new customers:

- Insert directly as active records.

9. Hive Limitation and Workaround

Problem

Hive non-transactional tables do not support:

- UPDATE
- DELETE

operations in a practical way.

Workaround Strategy

Instead of updating rows directly, I rebuilt the dimension table using:

```
INSERT OVERWRITE  
UNION ALL
```

This approach is commonly used in Hive batch processing environments.

10. SCD Type 2 Processing Logic

The SCD solution was implemented using four logical paths.

10.1 Preserve Historical Records

All already expired records (`is_current = 0`) were preserved.

10.2 Preserve Unchanged Current Records

Current records that did not change remained active.

10.3 Expire Changed Current Records

When a customer record changed:

- `end_date` was set to current date.
- `is_current` was changed to 0.

10.4 Insert New and Updated Records

Finally:

- New customers were inserted.
- Updated customer versions were inserted as active rows.

```
hive> SELECT customer_id, email, start_date, end_date, is_current FROM customer_dim_new WHERE customer_id = 12364;
OK
12364 upd_meyerjennifer@example.com 4/1/2025 2026-05-09 0
12364 meyerjennifer@example.com 2026-05-09 NULL 1
Time taken: 0.187 seconds, Fetched: 2 row(s)
hive> set hive.execution.engine=mr;
Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
hive> set mapreduce.framework.name=local;
hive> SELECT is_current, COUNT(*) FROM customer_dim_new GROUP BY is_current;
Query ID = itversity_20260509235021_f6ab9375-fbb2-487a-a8a0-3f8ccf6923a9
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks not specified. Estimated from input data size: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Job running in-process (local Hadoop)
2026-05-09 23:50:23,542 Stage-1 map = 100%, reduce = 100%
Ended Job = job_local1409253329_0001
MapReduce Jobs Launched:
Stage-Stage-1: HDFS Read: 3643852 HDFS Write: 82003990 SUCCESS
Total MapReduce CPU Time Spent: 0 msec
OK
0 131
1 4459
Time taken: 1.908 seconds, Fetched: 2 row(s)
```

11. Problems Faced During Implementation

During the implementation process, several technical issues occurred.

11.1 Hive Metastore Connection Failure

Problem

Hive failed with:

```
Unable to instantiate SessionHiveMetaStoreClient
```

Root Cause

The PostgreSQL metastore password did not match the Hive configuration.

Solution

I inspected:

```
grep -A1 "javax.jdo.option.ConnectionPassword"
```

Then updated Docker PostgreSQL credentials to match Hive configuration.

11.2 HDFS Connection Refused

Problem

HDFS initially failed with:

```
Connection refused
```

Root Cause

HDFS services were not started automatically.

Solution

I manually started HDFS using:

```
start-dfs.sh
```

and verified services using:

```
jps
```

11.3 MapReduce Execution Failures

Problem

Some Hive queries failed during MapReduce execution.

Root Cause

The lightweight Docker environment had issues running distributed MapReduce jobs.

Solution

I switched Hive execution to local mode:

```
set hive.execution.engine=mr;  
set mapreduce.framework.name=local;
```

This allowed Hive to execute jobs locally inside the container.

11.4 Permission Issues During Export

Problem

`hdfs dfs -getmerge` initially failed with permission errors.

Solution

I removed the old output file and recreated it before exporting the final result.

12. Final Output

The final generated file:

`output/customer_scd2_mixed.csv`

contained:

- Historical records
- Current records
- Expired records
- Newly inserted records

The final output successfully demonstrated:

- Change tracking
- History preservation
- SCD Type 2 implementation

```
itversity@hive_assignment:~$ head -20 /opt/hive_output/customer_scd2_mixed.csv
12346,Debra Wells,michaelkerr@example.net,3317035462,USNW Olsen FPO AA 43254,2025-02-10,2026-05-09,\N,1
12347,Michael Grimes,rchandler@example.net,243.521.3090,36923 Bowers Gateway Suite 027 New Kristi, MP 44312,2025-03-14,2026-05-09,\N,1
12348,Jennifer Cuevas,brandonbrown@example.com,+1-863-584-5200x273,095 Stevenson Lodge New Brettville, LA 67557,2025-01-03,2026-05-09,\N,1
12349,Anthony Harvey,dgonzales@example.net,771-305-3764x583,549 Taylor Mews Baileyburgh, NJ 59083,2025-04-01,2026-05-09,\N,1
12350,Thomas Martin,longpatricia@example.net,558-648-9504x3880,294 Patricia Light Suite 377 Marymouth, GA 49741,2025-02-07,2026-05-09,\N,1
12352,James Bush,dennis40@example.net,+1-650-456-4394x097,44332 Janet Cliffs Apt. 178 Johnsonfort, WA 55056,2025-03-02,2026-05-09,\N,1
12353,William Greene,floreschristina@example.net,001-609-772-9593x4809,5563 Brian Drives New Charlotteland, NY 81878,2025-03-15,2026-05-09,\N,1
12354,Emily Tanner MD,anna45@example.com,(886)301-5618,020 Willie Springs Apt. 266 Sueberg, GA 65464,2025-01-29,2026-05-09,\N,1
12355,James Acevedo,josephbrown@example.com,339-695-7693,1347 Taylor Station Suite 362 South Scottport, NH 32618,2025-02-24,2026-05-09,\N,1
12356,Melinda Kennedy,mary87@example.net,+1-420-680-5658x92614,527 Hunter Lane Apt. 717 Phillipstown, MS 10123,2025-01-19,2026-05-09,\N,1
12357,Sherri Mora,christophercook@example.com,458.978.5946,16245 Jason Rest Suite 741 Nortonmouth, MA 47922,2025-01-19,2026-05-09,\N,1
12358,Lucas Wood,george41@example.com,(579)527-3515x6724,922 Bell Valleys Apt. 515 Hansenmouth, DC 09410,2025-03-27,2026-05-09,\N,1
12359,Alexandra Smith,hensonjames@example.com,403.399.4781,4707 Mendoza Shoal Suite 365 New Gregory, IL 47589,2025-03-06,2026-05-09,\N,1
12360,Sean Anderson,powens@example.org,839-365-2193,04795 Hays Forges Suite 301 Millerburgh, NM 47791,2025-02-01,2026-05-09,\N,1
12361,Jeanette Stark,wfrancis@example.org,001-263-530-4234x4700,4209 Guzman Burg South Cynthia, KS 85592,2025-02-23,2026-05-09,\N,1
12362,Albert Bishop,baileynicole@example.com,315-873-7701x282,617 Brian Garden Simpsonhaven, RI 49804,2025-03-30,2026-05-09,\N,1
12363,Haley Sullivan,elizabethhardy@example.net,994-617-2939x962,664 Michael Turnpike Suite 930 Port Aprilmouth, PR 61100,2025-03-08,2026-05-09,\N,1
12364,Larry Hess,upd_meyerjennifer@example.com,001-626-971-5766,556 Schmidt Gateway Apt. 111 Kaneton, NM 59573,2/11/2025,4/1/2025,2026-05-09,0
12364,Larry Hess,meyerjennifer@example.com,001-626-971-5766,556 Schmidt Gateway Apt. 111 Kaneton, NM 59573,2025-02-11,2026-05-09,\N,1
12365,Nancy Shaw MD,greed@example.net,(561)740-1250,Unit 3498 Box 9802 DPO AE 01301,2025-02-16,2026-05-09,\N,1
itversity@hive_assignment:~$
```

13. Conclusion

This assignment provided practical experience with:

- Apache Hive
- Hadoop HDFS
- Dockerized Big Data environments
- Internal vs External tables
- CSV parsing challenges
- SCD Type 2 implementation
- Hive limitations and workarounds

One of the most important lessons learned was how batch-oriented Hive systems handle historical data tracking without relying on transactional updates.

I also gained hands-on experience troubleshooting real infrastructure problems related to:

- Hive Metastore
- Hadoop HDFS
- Permissions
- MapReduce execution

Overall, the assignment simulated a realistic data engineering workflow and improved my understanding of enterprise-scale data warehouse concepts.